

3-Year Systems + Hardware Roadmap

From C Fundamentals to GPU Infrastructure Engineering

Duration: 36 Months

Daily Commitment: 2–4h weekdays, 5–8h weekends

Progress Loop: Learn → Build → Measure → Document

Monthly Rule: Ship 1 deliverable + 1 write-up with measurements

Table of Contents

- 1. Overview & Philosophy
- 2. Operating Model (How to Execute)
- 3. Year 1: Systems Foundations (Months 1–12)
 - 3.1 Month-by-Month Overview
 - 3.2 Week-by-Week Detailed Calendar (Weeks 1–52)
- 4. Year 2: Infrastructure + Hardware Awareness (Months 13–24)
- 5. Year 3: GPU Specialization (Months 25–36)
- 6. Must-Finish Projects & Acceptance Criteria
- 7. Self-Testing & Grading
- 8. Job Search Timeline & Strategy
- 9. Resources & Tools Checklist
- 10. Quick Start: First 30 Days (Day-by-Day)

1. Overview & Philosophy

Core Principle: Think like the machine. Every abstraction you use has a cost in memory, CPU cycles, and latency. Your advantage is seeing the cost and making tradeoffs intentionally.

This roadmap is designed so you become someone who can:

- Read and reason about assembly and memory layout
- Profile and optimize performance using evidence (not opinions)
- Build low-latency, high-throughput infrastructure
- Understand hardware-facing constraints (cache, NUMA, I/O, PCIe)
- Specialize into GPU systems and orchestration by Year 3

Why this path works

Most people do	You will do	Result
Memorize APIs	Measure, profile, explain bottlenecks	Performance credibility
Build demos	Build systems with benchmarks + writeups	Portfolio that survives scrutiny
Study randomly	Follow weekly calendar + monthly deliverables	Consistent progress

2. Operating Model (How to Execute)

Daily time commitment

Day Type	Learning Time	What to do
Weekdays	2–4 hours	Reading + exercises. Do it before paid work.
Weekends	5–8 hours	Project work, profiling sessions, report writing.

Progress loop (every week)

1. **Learn:** read + summarize; collect “confusions”
2. **Build:** produce runnable code
3. **Measure:** benchmark + profile; capture logs
4. **Document:** README with decisions + results

Monthly validation

- **Deliverable:** one public repo update with reproducible scripts
- **Write-up:** 1–3 pages (what you built, what was slow, what you changed)
- **Self-test:** theory + practical (timed)

3. Year 1: Systems Foundations (Months 1–12)

Goal: think like the machine. Write safe C, understand memory and processes, profile and optimize with evidence, and ship a high-performance capstone.

Core resources (Year 1)

- CS:APP — primary spine
- OSTEP — OS foundations
- Brendan Gregg — performance tooling and mindset

3.1 Year 1: Month-by-Month Overview

Month	Theme	Deliverable	Proof
1	Source → assembly → binary → process	Annotated assembly + debugging report	Explain stack frames and crash root cause
2	Memory is real	Custom allocator + benchmark graphs	Leak/UAF detection logs + fragmentation story
3	Syscalls and processes	Mini shell (pipes + redirection)	Script test suite + syscall write-up
4	Profiling	Performance optimization report	Before/after flamegraphs + metrics table
5	Concurrency	Thread pool + contention study	Scaling curves + race fixes
6	Cache and micro- optimizations	Optimized matrix multiplication	Cache-miss counters and speedup graphs
7–12	Consolidation + capstone	High-performance KV server + interview material	Reproducible benchmarks + report + talk

3.2 Year 1: Week-by-Week Detailed Calendar (Weeks 1–52)

MONTH 1: From Source to Process

Reading: CS:APP Chapters 1–3

Week 1: Source → Running Process (Toolchain + Build Basics)

Total Time: 12–16 hours

Learning goals

- Understand the compilation pipeline (preprocess → compile → assemble → link)
- Set up a repeatable build + debug workflow
- Write and debug small C programs quickly

Daily tasks

- ☐ **Day 1 (2h):** Install toolchain and debugger. Verify `cc --version` and basic compile/run. Create repo and push initial structure (`src/`, `notes/`, `Makefile`).
- ☐ **Day 2 (2–3h):** Read CS:APP Ch.1. Write 2 tiny C programs. Compile with `-Wall -Wextra -O0 -g`. Step through in debugger.
- ☐ **Day 3 (2–3h):** Learn Make targets: `all`, `run`, `debug`, `clean`. Use it for your programs.
- ☐ **Day 4 (2h):** Inspect compilation stages: `-E`, `-S`, `-c`. Save outputs into `artifacts/`.
- ☐ **Day 5 (2–3h):** Create a multi-file program (`.h` + multiple `.c`). Link it. Inspect symbols with `nm` (or `otool` on macOS).
- ☐ **Weekend (6–8h):** Write `notes/week-01.md` explaining the build pipeline using your own artifacts, including “what is an object file?” and “what does the linker do?”

Deliverable

Repo with 5+ working C programs, `Makefile`, and a written explanation of the pipeline.

Week 2: Data Representation (Integers, Floats, Bits)

Total Time: 12–16 hours

Learning goals

- Understand signed vs unsigned, overflow, and two's complement
- Understand floating point precision and surprising behavior
- Get comfortable with bitwise operations as tools (not trivia)

Daily tasks

- ☐ **Day 1 (2–3h):** Read CS:APP Ch.2. Write overflow demos for signed/unsigned. Add assertions and explain results.
- ☐ **Day 2 (2–3h):** Implement bit utilities: set/clear/toggle bit, nth-bit, popcount. Add a minimal test runner.
- ☐ **Day 3 (2h):** Floating point demos: $0.1 + 0.2$, epsilon, catastrophic cancellation. Write short notes.
- ☐ **Day 4 (2–3h):** Implement hex dump tool (mini hexdump): reads file, prints offsets + hex + ASCII.
- ☐ **Weekend (6–8h):** Package utilities into `libbits` + tests. Add a README: “gotchas” you can explain in interviews.

Deliverable

Bit utility library + hex dump tool + write-up on integer/float gotchas.

Week 3: Machine-Level Programs (Reading Assembly)

Total Time: 14–18 hours

Learning goals

- Map C constructs to assembly patterns (loops, calls, conditionals)
- Understand stack frames, calling conventions, and registers
- Use disassemblers to explain what code does

Daily tasks

☐ **Day 1 (3h):** Read CS:APP Ch.3 basics. Learn prologue/epilogue and “where locals live”.

☐ **Day 2 (2–3h):** Compile with `-S`, study loops and if/else output. Compare `-O0` vs `-O2`.

☐ **Day 3 (2–3h):** Disassemble a binary (`objdump -d` on Linux, `otool -tv` on macOS). Label what each block does.

☐ **Day 4 (2h):** Practice: “given assembly, reconstruct C-like pseudocode”.

☐ **Weekend (8–10h):** Take a 100–150 line C program and produce a fully annotated assembly listing with stack frame diagrams.

Deliverable

Annotated assembly (with stack frame explanation) you can show to anyone.

Week 4: Month 1 Project + Self-Test

Total Time: 14–20 hours

Project

- ☐ **Day 1–2 (6h):** Write a C program (100+ LOC) using recursion, arrays, structs, pointers, multiple files.
- ☐ **Day 3 (4h):** Generate assembly for `-O0` and `-O2`. Create a “what changed and why” comparison.
- ☐ **Day 4 (3h):** Debug a crash end-to-end with debugger. Capture steps and the root cause.
- ☐ **Day 5 (2h):** Create an intentional memory bug (overflow) and explain failure at machine level.
- ☐ **Weekend (6–8h):** Write the Month-1 report: compilation pipeline + assembly notes + crash analysis.

Self-test (theory)

1. Explain what happens when you run a C program from source to execution.
2. Draw a stack frame for a function call with local variables.
3. Explain how a buffer overflow can corrupt a return address.

Acceptance criteria

- Working 100+ LOC C program pushed to GitHub
- Annotated assembly with stack frame notes
- Crash/root-cause writeup
- Self-test answers saved

MONTH 2: Memory Is Real

Reading: *CS:APP memory/performance* + *OSTEP virtual memory*

Week 5: Stack vs Heap (Memory Layout + Common Bugs)

Total Time: 12–16 hours

- ☐ **Day 1 (2–3h):** Print addresses of globals, stack locals, heap allocations; document layout.
- ☐ **Day 2 (2–3h):** Create leaks, double-free, and use-after-free examples; explain symptoms.
- ☐ **Day 3 (2h):** Add sanitizers (ASan/UBSan) where available; run and record output.
- ☐ **Day 4 (2h):** Study stack growth via recursion depth experiment.
- ☐ **Weekend (6–8h):** Write notes/week-05.md that teaches memory layout with your own screenshots/logs.

Week 6: Memory Debugging Tooling

Total Time: 12–16 hours

- ☐ **Day 1–2 (4–6h):** Run valgrind (Linux) or sanitizers (macOS) on intentionally buggy programs. Fix bugs and re-run.
- ☐ **Day 3 (2–3h):** Write a malloc wrapper that tracks allocations and prints a report at exit.
- ☐ **Weekend (6–8h):** Create a “bug zoo”: 6 small programs, each demonstrating a specific memory bug and its fix.

Week 7: Virtual Memory (Working Sets + Page Faults)

Total Time: 12–16 hours

- ☐ **Day 1–2 (4–6h):** Read VM chapters. Write a stride-based array walk that changes locality.
- ☐ **Day 3–4 (4–6h):** Measure impact: runtime changes as array size grows; record system counters if available.
- ☐ **Weekend (6–8h):** Write a VM experiment report explaining TLB, page faults, and your measurements.

Week 8: Project — Custom Allocator v1

Total Time: 14–22 hours

- ☐ **Day 1–2 (6h):** Implement a simple free-list allocator (malloc/free equivalents) with coalescing.
- ☐ **Day 3 (3–4h):** Add tests: random allocate/free patterns, fragmentation measurement.
- ☐ **Day 4–5 (4–6h):** Benchmark vs system malloc on a controlled workload. Plot results.
- ☐ **Weekend (6–8h):** Write README: design, tradeoffs, graphs, and what you'd do next.

Acceptance criteria

- Allocator passes tests
- Benchmarks vs malloc with graphs
- Fragmentation experiment described and reproducible

MONTH 3: Syscalls & Process Model

Reading: OSTEP processes + signals

Week 9: Processes (fork/exec/wait) + Tracing

Total Time: 12–16 hours

- ☐ **Day 1–2 (4–6h):** Write demos using fork, exec, waitpid. Explain exit codes.
- ☐ **Day 3–4 (4–6h):** Trace syscalls of common tools (ls, cat, cp). Summarize what you see.
- ☐ **Weekend (6–8h):** Write notes/week-09.md: process lifecycle and syscall patterns.

Week 10: Pipes + Redirection Building Blocks

Total Time: 12–16 hours

- ☐ **Day 1–2 (4–6h):** Build a two-process pipeline using pipe and dup2.
- ☐ **Day 3–4 (4–6h):** Implement basic redirection (>, <) in a minimal command runner.
- ☐ **Weekend (6–8h):** Write tests as shell scripts and run them automatically.

Week 11: Mini Shell v1

Total Time: 12–18 hours

- ☐ **Day 1–3 (6–9h):** Prompt loop + parsing + exec. Handle errors cleanly.
- ☐ **Day 4–5 (4–6h):** Add builtins: cd, exit. Add minimal job control notes (optional).
- ☐ **Weekend (6–8h):** Expand test scripts for correctness.

Week 12: Mini Shell v2 + Write-up

Total Time: 14–22 hours

- ☐ **Day 1–2 (4–6h):** Implement pipelines of arbitrary length.
- ☐ **Day 3–4 (4–6h):** Implement redirection robustly (multiple redirects).
- ☐ **Day 5 (2–3h):** Run full test suite; fix edge cases.
- ☐ **Weekend (6–8h):** 1-page report: syscalls used and why.

Acceptance criteria

- Shell passes script tests
- README explains syscalls and examples

MONTHS 4–6: Profiling, Concurrency, Cache (Weeks 13–24)

Reading: CS:APP optimization + concurrency sections; performance tooling materials

Weeks 13–14: Performance Measurement Tools

Total Time: 24–30 hours (2 weeks)

- ☐ **Week 13:** Baseline benchmarks for allocator/shell. Collect CPU time, RSS, and key counters (if on Linux, use `perf stat`).
- ☐ **Week 14:** Generate flamegraphs (Linux) or sampling profiles (macOS). Identify top hotspots and write hypotheses.
- ☐ **Deliverable:** baseline report + profiling artifacts committed.

Weeks 15–16: Optimization Project

Total Time: 24–32 hours (2 weeks)

- ☐ **Week 15:** Implement 1–2 optimizations with narrow scope. Re-profile and quantify impact.
- ☐ **Week 16:** Write a performance report: methodology, before/after table, and what you learned.

Acceptance criteria

- Measurable improvement or a correct explanation why not
- Reproducible benchmark commands

Weeks 17–18: Threads and Synchronization

Total Time: 24–30 hours (2 weeks)

- ☐ **Week 17:** Create race conditions on purpose, then fix using mutex/atomics. Write “what went wrong” notes.
- ☐ **Week 18:** Implement a thread pool library with a work queue, shutdown, and tests.

Weeks 19–20: Multithreaded Performance Project

Total Time: 24–32 hours (2 weeks)

- ☐ **Week 19:** Build a benchmark that exhibits lock contention. Measure throughput vs threads.
- ☐ **Week 20:** Improve scaling (sharding/finer locks). Produce scaling curves.

Acceptance criteria

- Thread pool repo
- Scaling curves and contention story

Weeks 21–22: Cache Architecture Deep Dive

Total Time: 24–30 hours (2 weeks)

- ☐ **Week 21:** Measure cache line size experimentally; implement stride experiments and plot results.
- ☐ **Week 22:** Demonstrate false sharing and fix with padding/alignment; measure before/after.

Weeks 23–24: Matrix Multiplication Optimization

Total Time: 26–34 hours (2 weeks)

- ☐ **Week 23:** Implement naive matmul; benchmark across sizes; capture counters if available.
- ☐ **Week 24:** Implement blocking/tiling and loop order changes; quantify speedup; write teaching-style report.

Acceptance criteria

- 5×+ speedup over naive (or best possible on your system explained)
- Cache-miss before/after measurements
- Graphs and reproducible benchmark scripts

MONTHS 7–12: Consolidation + Capstone (Weeks 25–52)

Focus: Build a production-quality portfolio piece

Months 7–12 consolidate everything into a flagship project: a high-performance KV server. You will benchmark it, profile it, optimize it, and write it up like a mini thesis.

Week 25: Sockets Basics (TCP Client/Server)

Total Time: 12–16 hours

- ☐ **Day 1–2:** Build TCP echo client/server. Handle partial reads/writes correctly.
- ☐ **Day 3–4:** Add simple protocol framing (length-prefix or line-delimited) and tests.
- ☐ **Weekend:** Benchmark localhost latency and throughput; record baseline.

Week 26: HTTP Basics (Build a Minimal HTTP Server)

Total Time: 12–18 hours

- ☐ **Day 1–2:** Learn HTTP request/response parsing; implement GET for static responses.
- ☐ **Day 3–4:** Implement header parsing + keep-alive basics; test with `curl`.
- ☐ **Weekend:** Write README describing protocol parsing and correctness constraints.

Week 27: Nonblocking I/O (select/poll)

Total Time: 12–18 hours

- ☐ **Day 1–3:** Convert server to nonblocking sockets; add `select` or `poll` event loop.
- ☐ **Day 4–5:** Add connection limits and timeouts; handle slow clients without blocking.
- ☐ **Weekend:** Benchmark vs blocking model; record CPU usage.

Week 28: Event Loop v2 (epoll/kqueue) + Month 7 Report

Total Time: 14–22 hours

- ☐ **Day 1–3:** Implement `epoll` (Linux) or `kqueue` (macOS/BSD) server loop.
- ☐ **Day 4–5:** Benchmark throughput/latency; include tail latency (p99).
- ☐ **Weekend:** Write Month 7 report: what model scaled best and why.

Week 29: Blocking vs Nonblocking I/O Deep Dive

Total Time: 12–18 hours

- ☐ **Day 1–2:** Create a load generator (or use `wrk` if HTTP). Define repeatable workload.
- ☐ **Day 3–5:** Compare models under load; capture latency percentiles and CPU usage.

Week 30: I/O Model Comparison (Write the Evidence)

Total Time: 12–18 hours

- ☐ **Day 1–3:** Create a single benchmark script that runs each server mode identically.
- ☐ **Day 4–5:** Write a comparison report with graphs and explanations.

Acceptance criteria

- Benchmark script is reproducible
- At least 3 I/O models compared
- Graphs include p50/p95/p99 and throughput

Week 31: `io_uring` (Optional, Linux) or Advanced Event Loop (macOS)

Total Time: 10–18 hours

- ☐ **Goal:** learn one advanced async I/O mechanism and document tradeoffs.
- ☐ **Deliverable:** prototype server + notes.

Week 32: Capstone Design Doc

Total Time: 10–16 hours

- ☐ **Day 1–3:** Write a 2–4 page design doc for the KV server: protocol, data model, concurrency model, metrics.
- ☐ **Day 4–5:** Define acceptance criteria (req/s, p99 targets, memory bounds) and a benchmark plan.

Capstone: High-Performance KV Server (Weeks 33–44)

Week 33: Core Data Structure

Total Time: 12–18 hours

- ☐ **Day 1–3:** Implement hash table (chaining or open addressing). Add correctness tests.
- ☐ **Day 4–5:** Add basic operations: GET/SET/DEL. Add fuzz-like random test.

Week 34: Protocol + Parser

Total Time: 12–18 hours

- ☐ **Day 1–2:** Define protocol framing (text or binary). Document it.
- ☐ **Day 3–5:** Implement parser and request validation; add negative tests.

Week 35: Server v1 (Single-thread Baseline)

Total Time: 12–20 hours

- ☐ **Day 1–3:** Implement event loop server handling multiple clients.
- ☐ **Day 4–5:** Add instrumentation: ops/sec, p50/p99, RSS.

Week 36: Benchmark Harness

Total Time: 12–18 hours

- ☐ **Day 1–2:** Build load generator (or adapt one). Define workloads: read-heavy, write-heavy, mixed.
- ☐ **Day 3–5:** Capture baseline graphs: throughput vs concurrency, p99 vs load.

Week 37: Profiling the Baseline

Total Time: 12–18 hours

- ☐ **Day 1–3:** Profile hotspots (CPU + allocation). Identify top 2 bottlenecks with evidence.
- ☐ **Day 4–5:** Write “bottleneck memo” describing hypotheses and next experiments.

Week 38: Optimization Round 1 (Parsing + Allocation)

Total Time: 12–20 hours

- ☐ **Day 1–3:** Reduce allocations (buffer reuse, arena, pooling). Keep changes measurable.
- ☐ **Day 4–5:** Re-benchmark and update graphs.

Week 39: Concurrency Model Choice

Total Time: 12–18 hours

- ☐ **Day 1–2:** Decide model: sharded maps, thread pool, or single-thread + batching.
- ☐ **Day 3–5:** Implement and test correctness under load.

Week 40: Contention + Tail Latency

Total Time: 12–20 hours

- ☐ **Day 1–3:** Measure contention and tail latency under load (p99/p999).
- ☐ **Day 4–5:** Apply one mitigation (sharding, backpressure, batching). Re-measure.

Week 41: Hardening (Limits, Timeouts, Failure Behavior)

Total Time: 10–18 hours

- ☐ **Day 1–3:** Add connection limits, timeouts, graceful shutdown, error handling.
- ☐ **Day 4–5:** Write overload behavior spec (what happens at $2\times$ intended load?).

Week 42: Benchmark Reproducibility

Total Time: 10–16 hours

- ☐ **Day 1–2:** Make benchmarks one-command reproducible (scripts + documented env).
- ☐ **Day 3–5:** Add a results folder with raw logs and plotted charts.

Week 43: Final Optimization Round

Total Time: 12–20 hours

- ☐ **Goal:** pick one “big win” optimization and validate across multiple runs (variance).
- ☐ **Deliverable:** final performance table (throughput + p50/p99 + CPU/mem).

Week 44: Final Report Draft (Design + Performance)

Total Time: 10–16 hours

- ☐ **Write:** a clear report: architecture, constraints, bottlenecks found, what worked, what didn't.

Month 12: Packaging + Job Prep (Weeks 45–52)

Week 45: README + Architecture Notes

Total Time: 10–16 hours

- ☐ **Write:** comprehensive README: architecture, API, build, benchmarks, results.

Week 46: Performance Report + Graphs

Total Time: 10–16 hours

- ☐ **Create:** graphs + optimization journey narrative; include flamegraphs.

Week 47: Release v1.0 + Cleanup

Total Time: 10–16 hours

- ☐ **Do:** code cleanup, testing pass, tag release, and freeze benchmarks.

Week 48: Resume + Interview Talk

Total Time: 10–16 hours

- ☐ **Write:** one-page resume with quantified bullets; prepare a 10–12 minute talk.

Week 49: Catch-up / Fix Weak Spots

Total Time: 8–16 hours

- ☐ **Goal:** close gaps from Months 1–11 without adding new scope.

Week 50: Deepen OS Knowledge (Self-Exam)

Total Time: 8–16 hours

☐ **Create:** 10-question OS/perf exam; answer without notes; fix weak topics.

Week 51: Small Bonus Project (Optional)

Total Time: 8–16 hours

☐ **Pick one:** tracer, log analyzer, small profiler, or load generator improvements.

Week 52: Portfolio Packaging

Total Time: 8–16 hours

☐ **Deliver:** a portfolio index: top 2–3 repos, quick summaries, how to run, results.

Year 1 capstone acceptance criteria

- KV server with reproducible benchmark scripts
- Latency percentiles (p50/p95/p99) reported under load
- At least one documented optimization with measured impact
- README and performance report suitable for interview discussion

4. Year 2: Infrastructure + Hardware Awareness (Months 13–24)

Goal: learn networking/storage/distributed systems deeply enough to build and measure real systems, while becoming hardware-aware (NUMA, PCIe/DMA concepts).

Months	Theme	Project	Proof
13–15	Networking internals	High-performance echo server (epoll/kqueue/io_uring)	Latency/throughput + traces
16–18	Storage systems	Append-only engine + compaction + fsync strategy	Write latency and amplification report
19–21	Distributed systems	Replicated KV (leader/follower or raft-backed)	Failure injection + correctness story
22–24	Hardware interface + NUMA	NUMA-aware tool or data structure	Placement report + improvements

Year 2: Month-by-Month Plan (Months 13–24)

Month	Focus	Build	Measure	Deliverable
13	TCP fundamentals + sockets mastery	TCP echo server/client + robust framing	Throughput/latency baselines + packet capture	Networking repo v1 + write-up
14	High-performance I/O (epoll/kqueue)	Event-loop server with backpressure	p50/p95/p99 under load + CPU/req	I/O server v2 + benchmark report
15	Zero-copy + advanced tuning	sendfile/zero-copy experiments (where available)	Tail latency study + mitigation	Networking phase report + graphs
16	Storage fundamentals (fsync, durability)	Append-only log writer + checksums	Write latency vs durability settings	Storage repo v1
17	Compaction + LSM-style design	Compaction prototype + index strategy	Write amplification measurement	Storage repo v2 + report
18	IO benchmarking discipline	Benchmark harness + workload generator	fio/blktrace/iostat experiments	Storage phase paper (2–3 pages)
19	Replication basics	Leader/follower KV replication	Failover correctness tests	Distributed KV v1
20	Consistency + partitions	Partitioning + membership changes	Latency under failure injection	Distributed KV v2 + experiments
21	Operational behaviors	Backpressure + load shedding + retries	Tail latency mitigation under chaos	Distributed phase report

22	NUMA + bandwidth awareness	NUMA experiment harness	Bandwidth/latency measurements	NUMA notes + baseline report
23	Tracing and profiling (eBPF optional)	Tracing scripts + profiling playbook	Find and explain 2 real bottlenecks	Tracing toolkit repo
24	Year-2 portfolio polish	Reproducible benchmarks + docs pass	Re-run full benchmark suite	Year-2 portfolio index + resume update

Year 2: Execution Calendar (Weeks 53–104)

Year 2 is detailed week-by-week for Months 13–24 (networking, storage, distribution, and hardware awareness).

Months 13–15: Networking Internals (Weeks 53–64)

Week 53: TCP Echo Server v1 (Correctness First)

Total Time: 12–18 hours

- ☐ **Goal:** a correct TCP client/server with robust I/O handling.
- ☐ **Day 1 (2–3h):** Implement server accept loop + per-connection read/write loop. Log errors and disconnect cleanly.
- ☐ **Day 2 (2–3h):** Implement client. Validate multi-message exchanges and multiple clients.
- ☐ **Day 3 (2–3h):** Fix partial read/write correctness (don't assume full buffers). Add "slow client" test.
- ☐ **Weekend (6–8h):** Add framing (newline-delimited or length-prefix). Add tests that send fragmented messages.
- ☐ **Deliverable:** runnable echo server/client + framing + tests.

Week 54: Measuring Networking Baselines

Total Time: 10–16 hours

- ☐ **Goal:** create a repeatable benchmark and learn what “good measurement” looks like.
- ☐ **Day 1 (2–3h):** Build a small load generator (N clients, fixed request size, fixed duration).
- ☐ **Day 2 (2–3h):** Capture throughput and latency percentiles (p50/p95/p99). Record environment.
- ☐ **Weekend (6–8h):** Run 5–10 repeated trials. Document variance and the chosen benchmark protocol.
- ☐ **Deliverable:** `bench/` scripts + baseline metrics table.

Week 55: Packet-Level Understanding (Trace + Explain)

Total Time: 10–16 hours

- ☐ **Goal:** explain what happens on the wire and tie it back to your code.
- ☐ **Day 1 (2–3h):** Capture a trace during a benchmark (Wireshark/tcpdump). Identify handshake and data packets.
- ☐ **Day 2 (2–3h):** Map trace events to your code path (accept, read loop, write loop, close).
- ☐ **Weekend (6–8h):** Write a 1–2 page note explaining the connection lifecycle and common pitfalls (Nagle, buffering, timeouts).
- ☐ **Deliverable:** trace artifacts + explanation note.

Week 56: Hardening (Timeouts, Limits, Failure Modes)

Total Time: 12–18 hours

- ☐ **Day 1–2 (4–6h):** Add timeouts, connection limits, and graceful shutdown.
- ☐ **Day 3 (2–3h):** Add “abuse tests” (slowloris-style, malformed frames, disconnect mid-frame).
- ☐ **Weekend (6–8h):** Re-run baselines and document changed behavior under overload.

Month 13 acceptance criteria

- Correct echo server with framing and tests
- Benchmark harness + baseline table
- Trace + written explanation

Week 57: Nonblocking I/O Baseline (select/poll)

Total Time: 12–18 hours

- ☐ **Day 1–3 (6–9h):** Convert server to nonblocking sockets. Implement `select` or `poll` loop.
- ☐ **Day 4–5 (4–6h):** Handle partial writes and output buffering correctly. Add tests for slow receivers.
- ☐ **Deliverable:** nonblocking server mode + tests.

Week 58: High-Performance Event Loop (epoll/kqueue)

Total Time: 12–20 hours

- ☐ **Day 1–3 (6–10h):** Implement epoll (Linux) or kqueue (macOS/BSD) event loop.
- ☐ **Day 4–5 (4–6h):** Add per-connection state machine and bounded buffers.
- ☐ **Deliverable:** event-loop server mode + correctness tests.

Week 59: Backpressure + Tail Latency

Total Time: 10–18 hours

- ☐ **Day 1–2 (4–6h):** Implement backpressure (drop/limit/queue) when buffers grow.
- ☐ **Day 3–5 (4–8h):** Benchmark tail latency (p99/p999) under bursty load. Document what improves or worsens.
- ☐ **Deliverable:** tail latency note + graphs.

Week 60: Comparative Benchmark Report (Blocking vs Nonblocking vs Event Loop)

Total Time: 10–16 hours

- ☐ **Day 1–2 (4–6h):** Create a single benchmark script that tests each I/O model identically.
- ☐ **Weekend (6–8h):** Write the report: metrics table, graphs, and explanations.

Month 14 acceptance criteria

- At least 3 I/O models implemented
- Reproducible benchmark script
- Graphs showing throughput + p50/p95/p99

Week 61: Zero-Copy Experiment Setup

Total Time: 10–16 hours

- ☐ **Goal:** understand where copies happen and how to measure them.
- ☐ **Day 1–2 (4–6h):** Identify a workload that transfers data (file → socket). Build baseline buffered path.
- ☐ **Day 3–5 (4–8h):** Implement sendfile-like path if supported; otherwise do “copy reduction” (larger buffers, fewer syscalls) and document limitations.

Week 62: Measure and Explain Copy Costs

Total Time: 10–16 hours

- ☐ **Day 1–2 (4–6h):** Benchmark throughput and CPU usage for baseline vs improved path.
- ☐ **Weekend (6–8h):** Write a short “where copies happen” explanation tied to your results.

Week 63: Burst Handling + Queueing

Total Time: 10–18 hours

- ☐ **Day 1–3 (6–9h):** Add bursty load scenario and observe queues/backpressure behavior.
- ☐ **Day 4–5 (4–6h):** Add one mitigation (bounded queues, early drop, prioritization) and re-measure.

Week 64: Networking Phase Paper

Total Time: 10–16 hours

- ☐ **Write:** 2–4 pages: architecture, I/O models, results, tail latency story, what you’d do next.

Month 15 acceptance criteria

- Zero-copy or copy-reduction experiment documented
- Tail latency improvements with graphs
- Networking phase paper shipped

Months 16–18: Storage Systems (Weeks 65–76)

Week 65: Record Format + Append-Only Log

Total Time: 12–18 hours

- ☐ **Day 1–2 (4–6h):** Define record format (header, key, value, checksum). Document it.
- ☐ **Day 3–5 (4–8h):** Implement append-only writer and reader, verify checksums, handle corruption cases.
- ☐ **Deliverable:** storage engine v1: log writer/reader + tests.

Week 66: Durability Controls (fsync, batching)

Total Time: 10–18 hours

- ☐ **Day 1–2 (4–6h):** Add durability modes (fsync per op, per batch, periodic).
- ☐ **Weekend (6–8h):** Benchmark write latency across durability modes and record results.

Week 67: Index Strategy v1

Total Time: 10–16 hours

- ☐ **Day 1–3 (6–9h):** Add an index (in-memory hash map from key → latest offset).
- ☐ **Day 4–5 (4–6h):** Support rebuild index from log on startup and test crash recovery.

Week 68: Month 16 Report (Durability Tradeoffs)

Total Time: 8–14 hours

☐ **Write:** 1–2 pages explaining fsync semantics, batching tradeoffs, and your measured results.

Month 16 acceptance criteria

- Append-only store with record format + checksums
- Durability modes implemented and benchmarked
- Crash recovery story (index rebuild) documented

Week 69: Compaction Design (What to Keep, What to Drop)

Total Time: 10–16 hours

☐ **Day 1–2 (4–6h):** Define compaction goals: reduce log size, remove stale records, preserve correctness.

☐ **Day 3–5 (4–8h):** Implement a compaction pass that rewrites live keys into a new segment.

Week 70: Write Amplification Measurement

Total Time: 10–16 hours

☐ **Day 1–2 (4–6h):** Create workloads (update-heavy, append-heavy, mixed) and run compaction.

☐ **Weekend (6–8h):** Measure amplification: bytes written / bytes of logical writes. Plot it.

Week 71: Read Path and Latency

Total Time: 10–18 hours

- ☐ **Day 1–3 (6–9h):** Optimize read path (index lookups, buffering, fewer syscalls).
- ☐ **Day 4–5 (4–6h):** Benchmark read latency before/after and record results.

Week 72: Month 17 Report (Compaction Tradeoffs)

Total Time: 8–14 hours

- ☐ **Write:** compaction design note + amplification graphs + correctness constraints.

Month 17 acceptance criteria

- Compaction implemented and tested
- Write amplification measured and explained
- Read latency benchmarks documented

Week 73: Benchmark Harness (Reproducibility)

Total Time: 10–16 hours

- ☐ **Day 1–2 (4–6h):** Create benchmark scripts: workloads, durations, warmup, output format.
- ☐ **Day 3–4 (4–6h):** Save raw logs and produce a simple summary table. Record environment in every run.

Week 74: System Tools Study (fio/iostat/blktrace)

Total Time: 8–14 hours

- ☐ **Day 1–2 (4–6h):** Run fio workloads to understand latency vs queue depth; document results.
- ☐ **Weekend (4–8h):** Use iostat and (if available) blktrace to interpret I/O behavior.

Week 75: Strategy Comparison (sync/async, batching, fsync)

Total Time: 10–16 hours

- ☐ **Day 1–3 (6–9h):** Run controlled comparisons across durability and batching strategies.
- ☐ **Day 4–5 (4–6h):** Summarize results into 2–3 key takeaways tied to evidence.

Week 76: Storage Phase Paper

Total Time: 8–14 hours

- ☐ **Write:** 2–3 page storage report (durability tradeoffs, amplification, and benchmark methodology).

Month 18 acceptance criteria

- Benchmark harness is reproducible
- System tools used and results interpreted
- Storage phase paper shipped

Week 77: Replication Log + Leader Append

Total Time: 12–18 hours

- ☐ **Build:** replicate writes via an append-only replication log (single leader).
- ☐ **Define:** request IDs and an idempotency strategy for retries.
- ☐ **Deliverable:** leader that ships a log stream to one follower.

Week 78: Follower Apply + Catch-Up

Total Time: 12–18 hours

- ☐ **Build:** follower apply loop with ordering guarantees and a durable “applied offset”.
- ☐ **Test:** crash/restart follower; verify correct catch-up behavior.
- ☐ **Deliverable:** follower replays from offset and reaches leader state.

Week 79: Client Semantics (Reads, Writes, and Consistency)

Total Time: 10–16 hours

- ☐ **Define:** read consistency model (leader reads, follower reads with staleness bounds, or explicit “read-your-writes”).
- ☐ **Build:** client retry rules + timeouts that don’t break correctness.
- ☐ **Deliverable:** client library or CLI that exercises failure/retry paths.

Week 80: Failover Demo + Correctness Tests

Total Time: 12–18 hours

- ☐ **Test:** duplicate requests, crash mid-write, follower lag, reconnect storms.
- ☐ **Demo:** kill leader, promote follower (manual is fine), and continue serving traffic.

Month 19 acceptance criteria

- Leader/follower replication working end-to-end
- Correctness tests cover crashes and duplicates
- Failover demo is repeatable

Week 81: Partitioning Strategy + Key Ownership

Total Time: 10–16 hours

- ☐ **Design:** partitioning approach (hash ring, fixed shards, or range).
- ☐ **Build:** routing layer that maps key → owner node.
- ☐ **Deliverable:** multi-node routing with one shard per node.

Week 82: Membership Changes (Join/Leave)

Total Time: 10–16 hours

- ☐ **Build:** membership config (static file or coordinator) and a safe update mechanism.
- ☐ **Test:** join/leave sequences; verify routing updates without serving wrong data.
- ☐ **Deliverable:** membership change demo with logs.

Week 83: Rebalancing + Data Movement

Total Time: 12–18 hours

- ☐ **Build:** rebalancing path (copy shard data, cut over ownership, clean up).
- ☐ **Test:** rebalancing correctness (no lost keys, no duplicates) under retries.
- ☐ **Deliverable:** rebalancing runbook + reproducible script.

Week 84: Latency Under Failures + Report

Total Time: 10–16 hours

- ☐ **Measure:** p50/p95/p99 latency during membership change and node failure.
- ☐ **Write:** experiment notes and an interpretation of the bottlenecks.

Month 20 acceptance criteria

- Partition routing and membership updates implemented
- Rebalancing path demonstrated
- Failure-injection latency measured and explained

Week 85: Backpressure + Load Shedding Policies

Total Time: 10–16 hours

- ☐ **Build:** bounded queues, overload responses, and retry hints.
- ☐ **Measure:** throughput and tail latency under bursty load.
- ☐ **Deliverable:** overload behaviors documented with graphs.

Week 86: Chaos Harness (Delay, Drop, Partition)

Total Time: 10–16 hours

- ☐ **Build:** fault injection controls (request delay, packet loss simulation, node kills).
- ☐ **Test:** correctness under chaos (no invalid reads, idempotent writes).
- ☐ **Deliverable:** chaos scripts + a repeatable scenario list.

Week 87: Tail Latency Mitigations

Total Time: 10–16 hours

- ☐ **Implement:** timeouts, circuit breakers, and at least one mitigation (hedged requests or adaptive batching).
- ☐ **Measure:** p99/p999 improvements under a fixed chaos scenario.
- ☐ **Deliverable:** before/after results with explanation.

Week 88: Distributed Phase Report

Total Time: 8–14 hours

- ☐ **Write:** 2–4 pages: failure model, mitigations, and measured results.

Month 21 acceptance criteria

- Chaos harness exists and is repeatable
- At least one mitigation measurably helps tail latency
- Distributed phase report shipped

Week 89: NUMA and Locality Baselines

Total Time: 10–16 hours

- ☐ **Measure:** memory bandwidth/latency baselines across thread counts and access patterns.
- ☐ **Document:** what “local” vs “remote” memory means for your target environment.
- ☐ **Deliverable:** baseline results table + notes.

Week 90: Pinning + Placement Harness

Total Time: 10–16 hours

- ☐ **Build:** harness to pin threads/cores and control memory allocation where possible.
- ☐ **Measure:** performance delta for “good” vs “bad” placement on one workload.
- ☐ **Deliverable:** scripts that reproduce the experiment.

Week 91: Apply Locality Lessons to Your KV

Total Time: 10–16 hours

- ☐ **Change:** one structural thing (sharding by thread, per-core queues, or data layout) to improve locality.
- ☐ **Measure:** throughput and tail latency before/after at high concurrency.
- ☐ **Deliverable:** short note linking change → locality effect.

Week 92: NUMA Report

Total Time: 8–14 hours

- ☐ **Write:** experiment design, results, and “what I would do in production”.

Month 22 acceptance criteria

- Locality harness exists and is reproducible
- One KV change tied to measured locality effect
- NUMA report shipped

Week 93: CPU Profiling Playbook

Total Time: 10–16 hours

- ☐ **Build:** profiling scripts + a repeatable “capture → interpret → act” checklist.
- ☐ **Deliverable:** profiling playbook repo folder you can reuse.

Week 94: Syscall + Network Tracing

Total Time: 10–16 hours

- ☐ **Capture:** syscall traces and one network trace for your distributed KV under load.
- ☐ **Explain:** which syscalls dominate and why (tie back to code paths).
- ☐ **Deliverable:** trace artifacts + explanation note.

Week 95: Bottleneck Case Study #1

Total Time: 12–18 hours

- ☐ **Find:** a real bottleneck in a prior project using your playbook.
- ☐ **Fix:** implement an improvement and measure before/after.
- ☐ **Deliverable:** case study write-up with numbers.

Week 96: Bottleneck Case Study #2 + Toolkit Release

Total Time: 12–18 hours

- ☐ **Repeat:** a second case study with a different bottleneck class (CPU, I/O, contention, network).
- ☐ **Package:** tracing/profiling toolkit into a clean repo directory with scripts and example outputs.

Month 23 acceptance criteria

- Profiling playbook exists and is reusable
- Two bottlenecks found and explained with evidence
- Toolkit packaged for reuse

Week 97: Benchmark Audit + Reproducibility Checklist

Total Time: 8–14 hours

- ☐ **Audit:** benchmark scripts and configs; ensure they capture environment and raw logs.
- ☐ **Fix:** one reproducibility issue (nondeterminism, missing flags, missing warmup).
- ☐ **Deliverable:** reproducibility checklist applied to all projects.

Week 98: Documentation and Script Cleanup

Total Time: 8–14 hours

- ☐ **Do:** tighten project entrypoints, add clear run commands, and standardize output locations.
- ☐ **Deliverable:** consistent repo structure across projects.

Week 99: Full Benchmark Re-Run

Total Time: 10–16 hours

- ☐ **Run:** a clean end-to-end benchmark pass (networking, storage, distributed, locality).
- ☐ **Summarize:** key metrics in a portfolio table.
- ☐ **Deliverable:** locked benchmark snapshots.

Week 100: Portfolio Index v1

Total Time: 8–14 hours

- ☐ **Build:** a single “portfolio index” page listing projects and results (links + numbers).
- ☐ **Deliverable:** portfolio index v1.

Week 101: Resume Rewrite With Quantified Bullets

Total Time: 6–12 hours

- ☐ **Do:** rewrite bullets to include metrics, scale, and evidence links.
- ☐ **Deliverable:** resume v1 + bullet bank.

Week 102: Project Talk Outline + Deep Dive Prep

Total Time: 8–14 hours

- ☐ **Prepare:** 10–15 minute tech talk outline and 2 deep-dive storyboards.
- ☐ **Deliverable:** talk deck draft + deep dive notes.

Week 103: Application Sprint + Tracking

Total Time: 6–12 hours

- ☐ **Do:** apply to roles and keep a structured log (company, role, links, follow-ups).
- ☐ **Deliverable:** application log with weekly targets.

Week 104: Year 2 Retrospective + Final Package

Total Time: 6–12 hours

- ☐ **Write:** a short retrospective: biggest wins, lessons, and what you'd rebuild.
- ☐ **Deliverable:** year-2 portfolio index + interview prep pack.

Month 24 acceptance criteria

- Benchmarks rerun and results locked
- Portfolio index and resume refreshed
- Interview prep materials drafted

5. Year 3: GPU Specialization (Months 25–36)

Goal: become a credible candidate for GPU performance / runtime / infra roles. You'll ship 1–2 “industry-grade” projects with profiling and design docs.

Months	Theme	Project	Proof
25–28	CUDA fundamentals + profiling	Kernel suite (matmul, reduction, convolution)	Nsight before/after metrics
29–31	CPU↔GPU orchestration + scheduling	Scheduler/orchestrator simulator	Utilization + tail latency improvements
32–34	Production plumbing	Containerized GPU workload + telemetry	E2E latency/cost report
35–36	Final packaging + interviews	Polish 2–3 flagship repos + talk	Demo, scripts, and documentation

Year 3: Month-by-Month Plan (Months 25–36)

Month	Focus	Build	Measure	Deliverable
25	CUDA setup + fundamentals	Kernel sandbox (vector add, reduce)	Nsight: occupancy + memory throughput	GPU repo v1
26	Memory hierarchy	Tiled matmul baseline + shared memory	Before/after metrics + roofline thinking	Matmul report v1
27	Optimization patterns	Reduce + scan + convolution kernels	Register pressure + achieved bandwidth	Kernel suite report
28	Packaging project A	Benchmark harness + reproducibility	Variance across runs and configs	Project A polished
29	CPU↔GPU transfers + orchestration	Pinned vs pageable benchmarks	Transfer overhead and batching curves	Orchestration notes
30	Scheduling basics	Workload generator + queueing model	Utilization vs tail latency tradeoffs	Scheduler sim v1
31	Scheduler improvements	Policies (fair, priority, SLO-aware)	Demonstrate improvement under mixed workloads	Project B design doc + results
32	Production plumbing	Containerized GPU workload	End-to-end latency baseline	Deployable demo v1
33	Observability	Metrics + tracing + dashboards	Explain bottlenecks end-to-end	Telemetry report
34	Cost/performance tuning	Batching + caching strategies	Cost per request/inference (if applicable)	Project C report

35	Portfolio polishing	READMEs, scripts, Dockerfiles	Re-run and lock benchmark results	Portfolio pack
36	Interviews	10–15 min tech talk + resume	Mock interviews + deep dives	Job-ready package

Year 3: Execution Calendar (Weeks 105–156)

Reality rule: Year 3 is where you prove impact. Every month must end with a measurable profile report.

Week 105: CUDA Setup + Benchmark Harness Skeleton

Total Time: 12–18 hours

- ☐ **Setup:** build toolchain and a minimal “run benchmarks” entrypoint.
- ☐ **Build:** timing harness (warmup, repeats, percentile reporting).
- ☐ **Deliverable:** GPU sandbox repo with reproducible benchmark runner.

Week 106: Vector Add + Memory Bandwidth Baseline

Total Time: 10–16 hours

- ☐ **Build:** vector add kernel with correctness tests and parameterized sizes.
- ☐ **Measure:** bandwidth vs size; record baseline metrics and variance.
- ☐ **Deliverable:** baseline chart + notes on memory-bound behavior.

Week 107: Reduction Baseline + Validation

Total Time: 12–18 hours

- ☐ **Build:** reduction kernel baseline (multiple variants allowed).
- ☐ **Test:** randomized tests and edge cases; ensure deterministic results.
- ☐ **Deliverable:** reduction baseline integrated into harness.

Week 108: First Nsight Profile + Month 25 Report

Total Time: 10–16 hours

- ☐ **Profile:** capture occupancy, memory throughput, and kernel timeline for vector add and reduction.
- ☐ **Write:** a short “baseline profile” report with screenshots and interpretation.

Month 25 acceptance criteria

- Benchmark harness exists and is reproducible
- At least 2 kernels implemented with correctness tests
- Nsight baseline profile captured and explained

Week 109: Matmul Baseline (Naive)

Total Time: 12–18 hours

- ☐ **Build:** naive matmul kernel with correctness tests and multiple shapes.
- ☐ **Measure:** throughput across sizes; record baseline kernel time.
- ☐ **Deliverable:** baseline matmul integrated into harness.

Week 110: Shared-Memory Tiling

Total Time: 12–18 hours

- ☐ **Build:** tiled matmul variant using shared memory.
- ☐ **Measure:** before/after performance and a first-pass explanation.
- ☐ **Deliverable:** tiled version + comparison table.

Week 111: Parameter Sweeps + Micro-Optimizations

Total Time: 10–16 hours

- ☐ **Experiment:** block sizes, unrolling, and memory access patterns.
- ☐ **Profile:** collect Nsight metrics for top candidates and pick a winner.
- ☐ **Deliverable:** sweep script + chosen configuration justified.

Week 112: Roofline-Style Reasoning + Month 26 Report

Total Time: 10–16 hours

- ☐ **Write:** a matmul report tying kernel time to memory throughput/compute utilization.
- ☐ **Deliverable:** “why it helped” notes per optimization with evidence.

Month 26 acceptance criteria

- Matmul baseline and optimized variant implemented
- Nsight metrics captured for baseline and best variant
- Report explains improvements with evidence

Week 113: Convolution Baseline

Total Time: 12–18 hours

- ☐ **Build:** convolution baseline (direct or transformed) with tests.
- ☐ **Measure:** performance across input sizes and kernel sizes.
- ☐ **Deliverable:** baseline conv integrated into harness.

Week 114: Memory Coalescing + Shared Memory in Convolution

Total Time: 12–18 hours

- ☐ **Optimize:** improve access patterns and reduce redundant global loads.
- ☐ **Profile:** collect before/after metrics and attribute wins to specific changes.
- ☐ **Deliverable:** conv optimization notes with evidence.

Week 115: Scan/Prefix-Sum Kernel

Total Time: 12–18 hours

- ☐ **Build:** scan kernel baseline with correctness tests and batch support.
- ☐ **Measure:** throughput and scaling as input grows.
- ☐ **Deliverable:** scan baseline integrated into harness.

Week 116: Kernel Suite Comparative Report (Month 27)

Total Time: 10–16 hours

- ☐ **Write:** a suite report: baselines, best variants, and Nsight screenshots.

Month 27 acceptance criteria

- At least 3 kernels with optimized variants
- Nsight evidence captured and explained
- Comparative report shipped

Week 117: Benchmark Standardization + Result Format

Total Time: 10–16 hours

- ☐ **Build:** unified CLI for running all kernels with consistent flags and outputs.
- ☐ **Deliverable:** standardized results schema and saved raw outputs.

Week 118: Reproducibility Pass (Environment + Variance)

Total Time: 8–14 hours

- ☐ **Add:** environment capture (GPU model, driver, toolkit version) to every run.
- ☐ **Run:** repeated trials and record variance; define your default methodology.
- ☐ **Deliverable:** reproducible benchmark methodology note.

Week 119: Packaging (Docs, Examples, and Plots)

Total Time: 8–14 hours

- ☐ **Do:** clean docs and examples; ensure a reader can reproduce plots.
- ☐ **Deliverable:** polished Project A repo structure.

Week 120: Project A Release + Month 28 Report

Total Time: 10–16 hours

- ☐ **Write:** final Project A report with baseline vs optimized summary and Nsight evidence.

Month 28 acceptance criteria

- Kernel suite packaged with reproducible runner
- Results include variance and environment capture
- Project A report shipped

Week 121: Workload Model + Simulator Skeleton

Total Time: 10–16 hours

- ☐ **Build:** workload generator (arrivals, service times, priorities).
- ☐ **Build:** simulator skeleton that can execute a policy and emit metrics.
- ☐ **Deliverable:** runnable simulator baseline.

Week 122: Baseline Scheduling Policies

Total Time: 10–16 hours

- ☐ **Implement:** FIFO and round-robin (or fair sharing) policies.
- ☐ **Measure:** utilization and p95/p99 latency under mixed load.
- ☐ **Deliverable:** baseline policy comparisons.

Week 123: Metrics + Tail Latency Instrumentation

Total Time: 10–16 hours

- ☐ **Add:** queue time breakdown, service time, and tail latency percentiles.
- ☐ **Deliverable:** results dashboard output (CSV + plot script is fine).

Week 124: Month 29 Report (Baselines)

Total Time: 8–14 hours

- ☐ **Write:** baseline policy results and bottleneck interpretation.

Month 29 acceptance criteria

- Simulator runs multiple policies and emits metrics
- Tail latency percentiles computed under load
- Baseline report shipped

Week 125: SLO-Aware Scheduling

Total Time: 10–16 hours

- ☐ **Implement:** SLO-aware or deadline-aware policy (prioritize near-deadline jobs).
- ☐ **Measure:** SLO miss rate vs baseline under bursty load.
- ☐ **Deliverable:** policy comparison with SLO metrics.

Week 126: Admission Control + Backpressure

Total Time: 10–16 hours

- ☐ **Build:** admission control and queue bounds; define overload behavior.
- ☐ **Measure:** tail latency under overload and the tradeoffs you choose.
- ☐ **Deliverable:** overload experiments documented.

Week 127: Mixed Workloads + Policy Tuning

Total Time: 10–16 hours

- ☐ **Run:** mixed workload experiments (short/long jobs, priority skew, burst patterns).
- ☐ **Tune:** policy parameters based on evidence.
- ☐ **Deliverable:** tuning notes + results table.

Week 128: Month 30 Report (SLO/Tail Latency)

Total Time: 8–14 hours

- ☐ **Write:** what policy wins and why (tie to queueing effects).

Month 30 acceptance criteria

- SLO-aware policy implemented and evaluated
- Admission control behavior defined under overload
- Report explains tail latency tradeoffs

Week 129: Multi-Node or Multi-GPU Scheduling Model

Total Time: 10–16 hours

- ☐ **Extend:** model to multiple workers with heterogeneous speeds or resource limits.
- ☐ **Deliverable:** multi-worker simulator mode.

Week 130: Faults, Stragglers, and Recovery

Total Time: 10–16 hours

- ☐ **Inject:** worker failures and stragglers; define rescheduling behavior.
- ☐ **Measure:** throughput and tail latency under failure scenarios.
- ☐ **Deliverable:** failure experiments documented.

Week 131: Optimization Pass (Policy + Implementation)

Total Time: 10–16 hours

- ☐ **Optimize:** one real bottleneck in simulator or policy logic; show measured improvement.
- ☐ **Deliverable:** before/after benchmark evidence.

Week 132: Design Doc + Project B Release (Month 31)

Total Time: 10–16 hours

- ☐ **Write:** 3–5 page design doc and a results summary.

Month 31 acceptance criteria

- Multi-worker scheduling model supported
- Failure scenarios evaluated with tail-latency results
- Design doc and release shipped

Week 133: Containerize a GPU Workload

Total Time: 10–16 hours

- ☐ **Build:** container image and a single command to run workload + benchmarks.
- ☐ **Deliverable:** reproducible build/run scripts.

Week 134: Telemetry (Metrics + Basic Tracing)

Total Time: 10–16 hours

- ☐ **Add:** metrics for queue times, batch size, GPU time, and end-to-end latency.
- ☐ **Deliverable:** metrics outputs with a simple dashboard script.

Week 135: End-to-End Baseline + Load Generator

Total Time: 10–16 hours

- ☐ **Build:** load generator with fixed workloads and reproducible configs.
- ☐ **Measure:** baseline p50/p95/p99 E2E latency and throughput.
- ☐ **Deliverable:** baseline results and environment capture.

Week 136: Month 32 Report (Baseline Plumbing)

Total Time: 8–14 hours

- ☐ **Write:** baseline architecture and metrics interpretation.

Month 32 acceptance criteria

- Containerized workload runs reproducibly
- Metrics and baseline E2E results captured
- Baseline plumbing report shipped

Week 137: Batching Strategy

Total Time: 10–16 hours

- ☐ **Implement:** dynamic or fixed batching policy with clear limits.
- ☐ **Measure:** latency vs throughput tradeoff across batch sizes.
- ☐ **Deliverable:** batching experiment results.

Week 138: Caching and Memory Footprint

Total Time: 10–16 hours

- ☐ **Implement:** one cache (results, weights, or inputs) with measurable impact.
- ☐ **Measure:** memory usage and latency impact under realistic load.
- ☐ **Deliverable:** cache tradeoff notes with metrics.

Week 139: Cost/Performance Measurement Plan

Total Time: 8–14 hours

- ☐ **Define:** cost model inputs and “cost per request” methodology.
- ☐ **Deliverable:** cost/perf worksheet and measurement script hooks.

Week 140: Month 33 Report (Batching/Caching/Cost)

Total Time: 8–14 hours

- ☐ **Write:** batching/caching results and cost/perf interpretation.

Month 33 acceptance criteria

- Batching tradeoffs measured with tail latency
- Caching impact measured with memory cost
- Cost/perf methodology documented

Week 141: Reliability Behaviors (Retries, Timeouts)

Total Time: 10–16 hours

- ☐ **Add:** timeouts, retries, and backoff with correctness-safe semantics.
- ☐ **Measure:** behavior under injected slowdowns and partial failures.
- ☐ **Deliverable:** reliability experiment notes.

Week 142: Resource Management + Scaling Model

Total Time: 10–16 hours

- ☐ **Implement:** one scaling lever (worker count, batch limits, concurrency caps) and measure impact.
- ☐ **Deliverable:** scaling experiments with metrics.

Week 143: Runbook + Incident Drill

Total Time: 8–14 hours

- ☐ **Write:** runbook: “how to debug latency regression”, “how to reproduce load”.
- ☐ **Do:** one incident drill (inject failure, diagnose, mitigate) and record steps.
- ☐ **Deliverable:** runbook + incident write-up.

Week 144: Project C Report + Release (Month 34)

Total Time: 10–16 hours

- ☐ **Write:** end-to-end report with latency, throughput, and cost/perf summary.

Month 34 acceptance criteria

- Reliability behaviors evaluated with failure injection
- Scaling lever measured and documented
- Project C report shipped

Week 145: Choose Flagship Repos + Gap List

Total Time: 6–12 hours

- ☐ **Pick:** 2–3 flagship repos and list missing pieces (docs, scripts, results, clarity).
- ☐ **Deliverable:** prioritized gap list and timeline.

Week 146: Reproducibility + Benchmark Lock-In

Total Time: 8–14 hours

- ☐ **Run:** final benchmark suite for Projects A–C and store raw outputs.
- ☐ **Deliverable:** locked benchmark snapshots with environment metadata.

Week 147: Documentation Polish + Example Walkthroughs

Total Time: 8–14 hours

- ☐ **Do:** improve docs so a reader can run, reproduce, and understand results.
- ☐ **Deliverable:** polished READMEs and example outputs.

Week 148: Portfolio Pack v1 (Month 35)

Total Time: 8–14 hours

- ☐ **Assemble:** one-page portfolio sheet and links to evidence (plots, reports, Nsight screenshots).

Month 35 acceptance criteria

- Flagship repos are reproducible and benchmarked
- Documentation supports quick evaluation
- Portfolio pack assembled

Week 149: Tech Talk Draft

Total Time: 6–12 hours

- ☐ **Build:** 10–15 minute tech talk deck around one flagship project.
- ☐ **Deliverable:** talk deck v1.

Week 150: Deep Dive Notes (GPU + Systems)

Total Time: 6–12 hours

- ☐ **Prepare:** deep dives on kernel optimization and one infra component (scheduler or plumbing).
- ☐ **Deliverable:** Q&A notes and diagrams.

Week 151: Mock Interviews Round 1

Total Time: 6–12 hours

- ☐ **Do:** mock interviews focused on performance debugging and GPU fundamentals.
- ☐ **Deliverable:** weaknesses list and targeted practice plan.

Week 152: Iterate on Weak Areas + Re-Measure

Total Time: 6–12 hours

- ☐ **Fix:** one identified weakness and update artifacts (talk, notes, or a project result).
- ☐ **Deliverable:** improved materials + refreshed evidence links.

Week 153: Question Bank (Kernel + Orchestration)

Total Time: 6–12 hours

- ☐ **Build:** question bank for kernels, memory hierarchy, scheduling, and tail latency.
- ☐ **Deliverable:** study sheet with short answers.

Week 154: Mock Interviews Round 2

Total Time: 6–12 hours

- ☐ **Do:** second mock round and refine explanations for your top projects.
- ☐ **Deliverable:** improved talk + more concise deep dive narratives.

Week 155: Application Sprint + Outreach

Total Time: 6–12 hours

- ☐ **Do:** apply and do targeted outreach; track responses and follow-ups.
- ☐ **Deliverable:** application log with weekly targets.

Week 156: Final Review + Interview-Ready Package (Month 36)

Total Time: 6–12 hours

- ☐ **Finalize:** portfolio sheet, talk deck, and links to evidence for all flagship work.

Month 36 acceptance criteria

- Talk and deep dives are practiced and polished
- Portfolio evidence links are complete and current
- Interview-ready package assembled

6. Must-Finish Projects & Acceptance Criteria

Project	Acceptance criteria (minimum)
Mini Shell (Year 1)	• fork/exec, pipes, redirection • script test suite • README explains syscalls
Custom Allocator (Year 1)	• free-list allocator + tests • fragmentation experiment • bench vs malloc + graphs
Year-1 KV Server (Capstone)	• baseline + optimized version • p50/p95/p99 latency and throughput under load • flamegraphs + reproducible benches • design + performance report
Storage Engine (Year 2)	• append-only + compaction • fsync strategies compared • latency and write amplification report
GPU Kernel Suite (Year 3)	• multiple kernels optimized with Nsight evidence • writeup per optimization explaining why it helped

Templates

Weekly checklist template

Week __ (Dates: __ → __)

- ☐ Reading completed (chapters/sections)
- ☐ Exercises completed (list)
- ☐ Build task finished (what you built)
- ☐ Measurement completed (commands + results)
- ☐ Write-up done (link)

Project template

Field	Fill this in
Problem statement	1 paragraph describing what matters
Goals	3 measurable goals
Non-goals	what you explicitly will not do
Acceptance criteria	numbers: latency, throughput, memory, correctness
Environment	CPU/GPU/OS/compiler/tool versions
Benchmark plan	workloads, number of runs, variance handling
Profiling plan	tools + what you'll capture
Results	before/after + interpretation

7. Self-Testing & Grading

Each month: 2 parts

Part	Duration	Rule	Pass condition
Theory quiz	~2 hours	10 questions, no notes	8/10 clear and correct
Practical task	4–8 hours	measure/fix a real program	show improvement or correct explanation

8. Job Search Timeline & Strategy

When	What to apply for	How to position
Months 9–12	Systems/infra internships	Lead with capstone baseline + early perf wins
Months 20–24	Entry/junior systems roles	Show storage + distributed KV with failure tests
Months 30–36	GPU/performance/compute roles	Bring Nsight evidence + scheduler design doc

Resume rule

- One page
- Quantified bullets: “reduced p99 latency by 38% using X; measured across N runs”
- Link to 2–3 flagship repos with benchmarks

9. Resources & Tools Checklist

Tools

Category	Tools
Build	gcc/clang, make, cmake
Debug	gdb/lldb, sanitizers, valgrind (Linux)
Inspect	objdump, nm, readelf (Linux), otool (macOS)
Trace	strace/dtruss
Profile	perf + flamegraphs (Linux), Nsight (Year 3)
Benchmark	wrk, fio, iperf

Books (recommended)

- CS:APP
- OSTEP
- DDIA
- TCP/IP Illustrated
- Programming Massively Parallel Processors

10. Quick Start: First 30 Days (Day-by-Day)

Days	Focus	Concrete tasks	Output
1–3	Toolchain + repo setup	Install compiler/debugger; create repo; add Makefile; verify build/debug loop.	Repo skeleton + notes
4–10	CS:APP Ch.1	5 small C programs; compile stages; save artifacts; debugger walkthrough.	Week 1 deliverable shipped
11–17	CS:APP Ch.2	integer/float experiments; bit utilities; basic tests; hexdump tool.	Week 2 deliverable shipped
18–24	CS:APP Ch.3	annotate assembly; compare –00 vs –02; disassemble binaries.	Week 3 deliverable shipped
25–30	Month 1 project + self-test	build 100+ LOC program; crash analysis; month report and self-test answers.	Month 1 complete